

Guía de Estudio — Módulo II: Pruebas Unitarias

Laboratorio de Programación y Lenguajes (IF009)

Universidad Nacional de Tierra del Fuego — Instituto IDEI

Ciclo Lectivo 2026 — 1er Cuatrimestre

Tema: 02 — Pruebas Unitarias

Semanas: 4 y 5

Carga horaria: 12 horas (2 sesiones teóricas + 2 prácticas de 3 hs c/u)

Docente: Matías Gel

1. Introducción — ¿Por qué dedicamos dos semanas a pruebas?

Hasta ahora, cada vez que escribimos un programa, la forma de verificar que funcionaba era ejecutarlo manualmente: correr el script, mirar la salida, probar con algún dato y decidir “parece que anda”. Ese enfoque tiene un problema serio: solo probamos lo que se nos ocurre en el momento, y si el programa crece, ya no podemos recordar todos los escenarios que habría que volver a verificar cada vez que hacemos un cambio.

Las pruebas unitarias son la herramienta que resuelve ese problema. En lugar de verificar manualmente, escribimos código que verifica nuestro código. Suena redundante, pero la ventaja es enorme: esas verificaciones se pueden ejecutar automáticamente, miles de veces, en segundos. Cada vez que modificamos algo, corremos la batería de pruebas y en menos de un segundo sabemos si rompimos algo o no.

Este módulo se ubica en un punto estratégico de la cursada. En el Módulo I ya trabajaron con funciones, clases, herencia y excepciones en Python. Ahora, antes de avanzar hacia frameworks más complejos como Django (Módulo III), necesitamos incorporar la disciplina de escribir pruebas. La razón es práctica: Django tiene su propio sistema de testing que se basa directamente en `unittest`, el framework que van a aprender acá. Todo lo que practiquen en estas dos semanas se transfiere de forma directa al resto de la materia.

Pero hay una razón más de fondo, que va más allá de la materia. En la industria del software, un proyecto sin pruebas automatizadas es un proyecto con fecha de vencimiento. Tarde o temprano, algún cambio va a romper algo que funcionaba, nadie va a darse cuenta a tiempo, y el costo de arreglar ese error va a ser desproporcionado. Los tres casos históricos que vamos a ver en este módulo — un avión, una máquina médica y un cohete espacial — son la versión extrema de ese problema. Pero la lógica aplica igual para una aplicación web, un sistema de facturación o un trabajo práctico.

Referencia: Este módulo cubre los puntos del plan mínimo: *importancia de la detección oportuna de errores, clasificación y tipos de pruebas, Prueba de Unidad: definición y propósito, procedimiento para pruebas de unidad, y soporte del lenguaje Python para pruebas de unidad (unittest).*

2. Objetivos de aprendizaje

Al finalizar este módulo vas a poder:

1. **Explicar** por qué el testing de software es una actividad crítica en el desarrollo, usando ejemplos concretos de fallas históricas.
2. **Distinguir** entre pruebas de caja negra y caja blanca, y entre los cuatro niveles de la pirámide de testing (unidad, integración, sistema, aceptación).
3. **Escribir** pruebas unitarias en Python usando el framework `unittest` : crear clases de test, usar aserciones apropiadas, y organizar el código con fixtures (`setUp` , `tearDown`).
4. **Aplicar** el ciclo TDD (Red → Green → Refactor) para diseñar código guiado por tests.
5. **Verificar** el manejo de excepciones en el código bajo prueba usando `assertRaises` .
6. **Usar** `unittest.mock` para aislar dependencias externas en los tests (`MagicMock` , `patch`).
7. **Ejecutar** suites de pruebas desde la terminal con `python -m unittest` y `discover` .

Taxonomía de Bloom: Los objetivos cubren desde *Recordar* (conceptos de testing) hasta *Crear* (diseñar tests propios desde cero con TDD). (Ver Filminas [F-00] a [F-41])

3. Conceptos previos necesarios

Antes de arrancar con este módulo, necesitás tener claro lo siguiente del Módulo I:

Concepto	¿Dónde se vio?	¿Por qué lo necesitamos?
Variables, funciones, colecciones	Módulo I	Los tests verifican funciones y métodos
Clases y <code>__init__</code>	Módulo I	<code>unittest.TestCase</code> es una clase que heredamos
Herencia y <code>super()</code>	Módulo I	Nuestras clases de test heredan de <code>TestCase</code>
Excepciones (<code>try</code> / <code>except</code> / <code>raise</code>)	Módulo I	Testeamos que las excepciones se lancen correctamente
Type hints básicos	Módulo I	Usamos type hints en todo el código del módulo
GitHub Classroom y Codespaces	Módulos 0/I	Los TPs se entregan por GitHub con autograding

Si algo de esta tabla no te resulta familiar, es buen momento para repasar los apuntes del Módulo I antes de seguir.

4. Desarrollo teórico

4.1. Repaso de POO en Python — lo que necesitamos para unittest

(Ver Filminas [F-02] a [F-05])

Antes de entrar en pruebas, necesitamos repasar algunos conceptos de programación orientada a objetos que van a ser fundamentales. No es un capricho: `unittest.TestCase` es una clase, nuestras pruebas heredan de ella, y el código que vamos a testear (la clase `Agenda`) usa herencia, excepciones propias y métodos especiales. Si no tenemos fresco el modelo de objetos de Python, nos vamos a perder.

4.1.1. Clases, instancias y métodos especiales

Una clase en Python se define con la palabra clave `class` y tiene un constructor `__init__` que se ejecuta automáticamente al crear una instancia. Veamos la clase `Agenda` que vamos a usar como ejemplo a lo largo de todo el módulo:

```
class Agenda:
    def __init__(self, capacidad: int = 100) -> None:
        self._contactos: dict = {}
        self.capacidad = capacidad

    def __str__(self) -> str:
        return f"Agenda con {len(self._contactos)} contacto(s)"

    def __eq__(self, other: object) -> bool:
        return (isinstance(other, Agenda)
                and self._contactos == other._contactos)
```

Hay tres métodos especiales en este ejemplo que vale la pena entender:

- `__init__`: el constructor. Recibe los parámetros iniciales y configura el estado del objeto. Acá crea un diccionario vacío para los contactos y guarda la capacidad.
- `__str__`: define qué se muestra cuando hacemos `print(agenda)`. Sin este método, Python muestra algo como `<Agenda object at 0x7f...>`, que no dice nada útil.
- `__eq__`: define qué significa que dos agendas sean "iguales". Esto es crucial para los tests: cuando escribamos `self.assertEqual(agenda1, agenda2)`, Python va a llamar a `__eq__` internamente. Si no lo implementamos, `assertEqual` compara identidad de objetos (que sean el mismo objeto en memoria), no que tengan el mismo contenido.

Regla práctica: si vas a testear una clase, implementá `__eq__` y `__str__`. Invertir cinco minutos en eso te ahorra horas de debugging.

4.1.2. Herencia y excepciones propias

La herencia permite que una clase extienda a otra. El patrón que más nos importa acá es el de excepciones propias:

```
class Error(Exception):
    """Clase base para errores de la aplicación."""
    pass

class DNIInvalidoError(Error):
    """Se lanza cuando el DNI no tiene formato válido."""
    def __init__(self, dni: str) -> None:
        super().__init__(f"DNI inválido: '{dni}'. Debe tener 7 u 8 dígitos.")
        self.dni = dni
```

¿Por qué nos importa esto? Por dos razones:

1. **Para los tests:** vamos a necesitar `assertRaises(ValueError)` para verificar que el código lanza excepciones cuando corresponde. Si `DNIInvalidoError` hereda de `ValueError`, `assertRaises(ValueError)` la va a capturar también. Eso puede ser intencional o no — es una decisión de diseño que hay que tomar.
2. **Para entender TestCase:** la clase `unittest.TestCase` sigue el mismo patrón. Cuando escribimos `class AgendaTest(unittest.TestCase)`, estamos heredando de una clase que ya tiene toda la maquinaria de ejecución de tests, aserciones y fixtures. Nosotros solo agregamos los métodos `test_*`.

4.1.3. El código que vamos a testear

La clase `Agenda` con validaciones y excepciones:

```
class Agenda:
    def agregar(self, dni: str, nombre: str, apellido: str,
                direccion: str, telefono: str) -> None:
        if not dni.isdigit() or not (7 <= len(dni) <= 8):
            raise ValueError(f"DNI inválido: {dni!r}")
        if dni in self._contactos:
            raise KeyError(f"DNI {dni!r} ya registrado")
        self._contactos[dni] = {
            "nombre": nombre,
            "apellido": apellido,
            "direccion": direccion,
            "telefono": telefono,
        }
```

Fijate las dos validaciones:

- `not dni.isdigit()` atrapa letras, espacios, guiones — cualquier cosa que no sean dígitos puros.
- `not (7 <= len(dni) <= 8)` atrapa DNIs demasiado cortos o largos.

Cuando escribamos los tests, vamos a necesitar un caso de prueba para cada una de estas condiciones. Eso se llama **cobertura de ramas**: asegurarnos de que cada camino posible del código fue ejercitado al menos una vez.

4.1.4. Métodos especiales útiles para tests

Método	Cuándo implementarlo	Impacto en los tests
<code>__eq__</code>	Cuando <code>==</code> debe comparar estado interno	Permite usar <code>assertEqual(obj1, obj2)</code> comparando contenido
<code>__str__</code>	En cualquier clase de dominio	Mejora los mensajes de error cuando un test falla
<code>__repr__</code>	Para debugging	Aparece en las trazas de unittest cuando algo falla
<code>__len__</code>	En colecciones propias	Permite escribir <code>assertEqual(len(agenda), 2)</code>

4.2. Motivación: el costo de no testear

(Ver *Filminas [F-06]* a *[F-13]*)

Antes de meternos con el framework, veamos tres casos reales donde la falta de tests adecuados tuvo consecuencias catastróficas. No son anécdotas lejanas: son recordatorios de que el software controla sistemas críticos, y que “parece que funciona” no es una verificación aceptable.

4.2.1. Vuelo 140 de Air China — 1994

El 26 de abril de 1994, un Airbus A300 de Air China realizaba el vuelo 140 con destino a Nagoya, Japón. Durante la aproximación final, un error en el software del piloto automático provocó que el sistema cambiara de modo de manera inesperada, sin una señalización clara a la tripulación. La combinación de entradas que disparó el error nunca había sido testeada específicamente.

El resultado: el avión impactó contra el suelo. 264 personas murieron.

El patrón relevante para nosotros: el software “funcionaba” en todos los escenarios que habían sido probados. El problema estaba en una combinación de estados del sistema que nadie pensó en verificar. En términos de testing, faltaban **pruebas de borde** — casos donde múltiples condiciones se combinan de maneras poco obvias.

[python-testing.pdf — slide “Errores!!!”]

4.2.2. Therac-25 — 1985 a 1987

El Therac-25 era una máquina de radioterapia utilizada en hospitales de Estados Unidos y Canadá. Su versión anterior, la Therac-20, tenía bloqueos de hardware que impedían que el equipo emitiera dosis excesivas de radiación. Cuando diseñaron la Therac-25, eliminaron esos bloqueos físicos confiando en que el software los reemplazaría adecuadamente.

El problema: el software del Therac-25 reutilizaba código de la Therac-20, pero ese código asumía la existencia de los bloqueos de hardware. Bajo ciertas condiciones de carrera (*race condition*) — que ocurrían cuando el operador tipeaba rápidamente una secuencia específica de teclas — la máquina podía emitir hasta 100 veces más radiación de lo normal.

El resultado: al menos 4 muertes confirmadas y 3 heridos graves con quemaduras severas por radiación.

Lo más preocupante del caso: el error era intermitente. A veces ocurría y a veces no, dependiendo de la velocidad de tipeo del operador. Los logs de la máquina mostraban “MALFUNCTION 54” sin descripción clara. Tardaron dos años en identificar la causa raíz.

En términos de testing: este es un caso de **caja blanca**. La condición de carrera solo podía descubrirse conociendo la estructura interna del código y testeando combinaciones de timing que no eran obvias desde la especificación externa. Las pruebas de caja negra (probar entradas y verificar salidas) no habrían encontrado este bug.

[python-testing.pdf — slide “Therac-25”]

4.2.3. Ariane 5 — 1996

El 4 de junio de 1996, el cohete Ariane 5 de la Agencia Espacial Europea despegó en su vuelo inaugural desde la Guayana Francesa. 37 segundos después del despegue, se autodestruyó.

La investigación reveló la causa: el sistema de navegación del Ariane 5 reutilizaba el módulo de referencia inercial del Ariane 4. Ese módulo contenía una conversión de un número en punto flotante de 64 bits a un entero de 16 bits. En el Ariane 4, esa conversión nunca fallaba porque los valores del sensor estaban dentro del rango de 16 bits. El Ariane 5, al ser más rápido, generaba valores que excedían ese rango, provocando un **overflow**.

Lo irónico: el código que falló era un módulo de alineación que ni siquiera se usaba activamente durante el vuelo. Costo total: 370 millones de dólares en 37 segundos.

El análisis de seguridad del Ariane 4 decía textualmente: “Este valor no puede exceder el rango”. Correcto para el Ariane 4. Incorrecto para el Ariane 5.

En términos de testing: faltaron **pruebas de regresión** — re-ejecutar los tests existentes con los nuevos parámetros del Ariane 5 — y **pruebas de borde** con los valores máximos del nuevo entorno.

[python-testing.pdf — slide “Arian 5”]

4.2.4. ¿Qué tienen en común estos tres casos?

Los tres tenían software escrito por profesionales competentes. Los tres habían pasado por procesos de verificación. Los tres fallaron en producción porque **no se testearon los casos de borde ni el comportamiento bajo condiciones nuevas**.

La definición de testing que vamos a usar en este módulo viene del estándar IEEE:

“Es un proceso que busca verificar la exactitud, integridad y calidad de un software. Incluye una serie de actividades que buscan encontrar errores y fallas en el software previo a que lleguen al usuario.”

Hay un matiz importante en esa definición: dice “busca **encontrar** errores”, no “busca **confirmar** que funciona”. Testing es una actividad de falsificación. El objetivo no es demostrar que el código anda, sino descubrir las condiciones bajo las cuales falla.

4.3. Clasificación de pruebas

(Ver Filminas [F-11] a [F-13])

4.3.1. Según el conocimiento del sistema: caja negra vs. caja blanca

Hay dos enfoques fundamentales para diseñar pruebas, y se distinguen por cuánto sabemos (o queremos saber) sobre la implementación interna del código que estamos testeando:

Prueba de caja negra: se diseñan las entradas y se verifican las salidas únicamente contra la especificación, sin mirar el código fuente. Ejemplo: si la especificación dice “el método `agregar` lanza `ValueError` cuando el DNI contiene letras”, probamos con `dni = "abc"` y verificamos que se lance la excepción. No importa cómo está implementada la validación internamente.

Prueba de caja blanca: se mira el código fuente y se diseñan las entradas para ejercitar ramas específicas. Ejemplo: vemos que la condición es `not (7 <= len(dni) <= 8)`, entonces creamos un test con un DNI de exactamente 6 dígitos (justo por debajo del límite) y otro con 9 dígitos (justo por encima). Estos son los llamados *valores de frontera* o *boundary values*.

En la práctica, un buen conjunto de tests combina ambos enfoques. Los primeros tests suelen ser de caja negra (verificar el comportamiento contra la especificación), y luego se agregan tests de caja blanca para cubrir ramas y condiciones que no fueron alcanzadas.

[python-testing.pdf — slide “Testing de software” (caja negra/blanca)]

4.3.2. Según el nivel de granularidad: la pirámide de testing

Las pruebas se organizan en cuatro niveles, formando una pirámide donde la base es ancha (muchos tests rápidos) y la cima es angosta (pocos tests lentos):

Nivel	Nombre	¿Qué prueba?	Velocidad	Herramienta
1 (base)	Prueba de Unidad	Una clase o función aislada	Milisegundos	<code>unittest</code>
2	Prueba de Integración	Interacción entre módulos	Segundos	<code>unittest</code> + fixtures reales
3	Prueba de Sistema	El sistema completo desde afuera	Minutos	<code>unittest</code> + CLI / HTTP
4 (cima)	Prueba de Aceptación	Requisitos del usuario	Minutos/horas	Escenarios manuales o automatizados

En este módulo nos concentramos en el Nivel 1: Pruebas de Unidad. La definición de las filminas originales del curso es precisa:

“Son pruebas de bajo nivel (a nivel de código) que se focalizan en una pequeña parte del software. En programación orientada a objetos estas unidades suelen ser las clases.”

¿Por qué empezar por la base de la pirámide? Porque las pruebas unitarias son rápidas (se ejecutan en milisegundos), baratas (no necesitan infraestructura externa) y precisas (cuando fallan, señalan exactamente qué parte del código tiene el problema). Los niveles superiores son importantes, pero sin una base sólida de pruebas unitarias, todo lo demás se vuelve frágil.

[python-testing.pdf — slide “Test de unidades – Unit Test”]

4.3.3. Pruebas de regresión

Un concepto transversal a todos los niveles: una **prueba de regresión** consiste en re-ejecutar toda la batería de tests existentes después de cada modificación del código. Si todos pasan, el cambio no rompió nada. Si alguno falla, sabemos exactamente qué cambio causó el problema.

El Ariane 5 falló porque el código del Ariane 4 se reutilizó sin ejecutar los tests con los nuevos parámetros. Si hubieran corrido una batería de regresión con los valores de velocidad del Ariane 5, el overflow habría sido detectado antes del lanzamiento.

Con TDD (que veremos más adelante), las pruebas de regresión son automáticas: cada vez que ejecutamos `python -m unittest discover`, estamos haciendo regresión sobre todo el proyecto.

4.4. El framework unittest

(Ver Filminas [F-14] a [F-20])

4.4.1. Contexto histórico: la familia xUnit

Para entender por qué `unittest` tiene la estructura que tiene, conviene conocer su historia:

Año	Framework	Lenguaje	Creador
1989	SUnit	Smalltalk	Kent Beck
1997	JUnit	Java	Kent Beck + Erich Gamma
2001	unittest	Python	Steve Purcell (port de JUnit)
2002	NUnit	.NET	Charlie Poole

Kent Beck, que trabajaba en Smalltalk en los años 80, creó el primer framework de pruebas unitarias moderno: SUnit. La idea central era simple: encapsular cada prueba en un método de una clase, y que un “runner” las ejecute automáticamente y reporte los resultados. Ese patrón fue tan exitoso que se portó a prácticamente todos los lenguajes importantes.

`unittest` de Python es parte de la biblioteca estándar desde Python 2.1 — no hay que instalar nada. Y el patrón es exactamente el mismo que en JUnit o NUnit: clase base → herencia → métodos `test_*` →

discovery automático. Si entienden `unittest`, van a entender la lógica de testing en cualquier lenguaje de la familia xUnit.

[python-testing.pdf — slide "Unittest framework", "Xunit"]

4.4.2. Los tres conceptos centrales

El framework se organiza alrededor de tres abstracciones:

TestCase (caso de prueba): la unidad de organización de tests. Se crea heredando de `unittest.TestCase`. Cada método cuyo nombre empiece con `test_` es un caso de prueba independiente que el runner detecta y ejecuta automáticamente.

TestSuite (conjunto de pruebas): una colección de TestCases que se ejecutan juntos. Raramente se crea manualmente — el runner la construye automáticamente por discovery (buscando todos los archivos `test*.py` del proyecto).

TestRunner (ejecutor): el componente que orquesta la ejecución y muestra los resultados. `python -m unittest` es el runner por defecto. En un entorno de CI (como GitHub Actions), el runner lee el código de salida: 0 si todos pasan, 1 si alguno falla.

[python-testing.pdf — slide "Unittest – clase de hoy"]

4.4.3. Estructura mínima de un test

Veamos el caso más simple posible:

```
import unittest

class PrimeraVez(unittest.TestCase):

    def test_suma(self):
        resultado = 2 + 2
        self.assertEqual(resultado, 4)

    def test_tipo(self):
        self.assertIsInstance("hola", str)

if __name__ == "__main__":
    unittest.main()
```

Paso a paso:

1. Importamos `unittest`.
2. Creamos una clase que hereda de `unittest.TestCase`.
3. Definimos métodos que empiecen con `test_`. Cada uno es un caso de prueba independiente.
4. Dentro de cada método, usamos aserciones (`assertEqual`, `assertIsInstance`, etc.) para verificar condiciones.
5. `unittest.main()` al final permite ejecutar el archivo directamente con `python test_primera.py`.

Reglas del runner:

- Solo los métodos con nombre `test_*` se ejecutan automáticamente.
- Un método = un test = un resultado:  ok,  FAIL, o  ERROR.
- El orden de ejecución entre tests **no está garantizado**. Cada test debe ser independiente del resto.

4.4.4. Entendiendo la salida de un test fallido

Cuando un test falla, `unittest` muestra información muy específica. Es importante saber leerla:

FAIL — la aserción no se cumplió:

```
FAIL: test_nombre_correcto (test_agenda.AgendaTest)
AssertionError: 'Juana' != 'Juan'
```

Esto significa que `assertEqual` esperaba `'Juan'` pero recibió `'Juana'`. El error está en el código que genera el valor, no en el test.

ERROR — excepción inesperada:

```
ERROR: test_buscar_existente (test_agenda.AgendaTest)
KeyError: '12345678'
```

Esto significa que el código lanzó una excepción que el test no esperaba. Hay un bug en el método `buscar` (o en los datos del `setUp`).

Regla: empezá a leer siempre desde la línea del `AssertionError` o la excepción. Ahí está el diagnóstico.

Con la opción `-v` (verbose), la salida muestra el nombre completo de cada test:

```
test_agregar_nuevo ... ok
test_buscar_existente ... FAIL
test_dni_invalido ... ok

Ran 3 tests in 0.002s  FAILED (failures=1)
```

4.4.5. Aserciones: la herramienta fundamental

Las aserciones son los métodos que usamos dentro de los tests para verificar condiciones.

`unittest.TestCase` provee una colección completa. Las más usadas son:

Aserciones de igualdad y tipo:

Método	Qué verifica
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>bool(x) is True</code>
<code>assertFalse(x)</code>	<code>bool(x) is False</code>
<code>assertIsNone(x)</code>	<code>x is None</code>
<code>assertIsNotNone(x)</code>	<code>x is not None</code>
<code>assertIn(a, b)</code>	<code>a in b</code> (funciona con listas, dicts, strings)
<code>assertIsInstance(a, T)</code>	<code>isinstance(a, T)</code>
<code>assertRaises(Exc)</code>	se lanza la excepción <code>Exc</code>

Aserciones para colecciones:

Método	Qué verifica
<code>assertCountEqual(a, b)</code>	mismos elementos sin importar orden
<code>assertListEqual(a, b)</code>	listas iguales — muestra diff detallado
<code>assertDictEqual(a, b)</code>	diccionarios iguales — muestra diff detallado
<code>assertRegex(s, r)</code>	string <code>s</code> coincide con regex <code>r</code>

Un consejo importante: **usá siempre la aserción semántica más específica**. Si querés verificar igualdad, usá `assertEqual(a, b)`, no `assertTrue(a == b)`. La diferencia está en el mensaje de error:

- `assertEqual`: `AssertionError: 4 != 5` — te dice exactamente qué valores no coinciden.
- `assertTrue`: `AssertionError: False is not true` — no te dice nada útil. Tenés que adivinar qué salió mal.

[python-testing.pdf — slide “TestCase – sentencias assert”]

4.4.6. Testeando excepciones con `assertRaises`

Una necesidad muy frecuente es verificar que el código lanza una excepción bajo ciertas condiciones. Para eso existe `assertRaises`, que se usa preferentemente como *context manager* (con `with`):

```
def test_dni_invalido_lanza_error(self):
    with self.assertRaises(ValueError):
        Agenda().agregar("abc", "Juan", "Pérez", "Calle 1", "111")
```

¿Cómo funciona? El bloque `with` monitorea todo lo que ocurre dentro. Si `ValueError` se lanza, el test pasa. Si no se lanza, el test falla con `AssertionError: ValueError not raised`.

También podemos capturar la excepción y verificar su mensaje:

```
def test_mensaje_de_error_correcto(self):
    with self.assertRaises(ValueError) as ctx:
        Agenda().agregar("abc", "Juan", "Pérez", "Calle 1", "111")
    self.assertIn("abc", str(ctx.exception))
```

Esto es útil cuando queremos asegurarnos de que el mensaje de error sea descriptivo y contenga la información relevante (en este caso, el DNI inválido que se intentó usar).

[python-testing.pdf — slide “TestCase - Sentencias assert - excepciones”, “assertRaises”]

4.4.7. Fixtures: setUp y tearDown

Cuando tenemos varios tests que necesitan los mismos datos de partida, repetir la inicialización en cada método sería una violación del principio DRY (*Don't Repeat Yourself*). Los **fixtures** resuelven esto:

```
class AgendaTest(unittest.TestCase):

    def setUp(self):
        """Se ejecuta ANTES de cada test."""
        self.agenda = Agenda()
        self.agenda.agregar("12345678", "Juan", "Pérez", "Calle 1", "2901-111")

    def tearDown(self):
        """Se ejecuta DESPUÉS de cada test (incluso si falla)."""
        del self.agenda

    def test_buscar_existente(self):
        resultado = self.agenda.buscar("12345678")
        self.assertEqual(resultado["nombre"], "Juan")

    def test_buscar_inexistente(self):
        with self.assertRaises(KeyError):
            self.agenda.buscar("00000000")
```

Puntos clave:

- **setUp** se ejecuta **antes de cada test** — no una vez al principio, sino antes de cada `test_*`. Esto garantiza que cada test arranque con un estado limpio.
- **tearDown** se ejecuta **después de cada test**, incluso si el test falla. Es el lugar para liberar recursos (cerrar conexiones, borrar archivos temporales).
- Los tests **no deben depender del orden de ejecución**. Cada test debe funcionar correctamente sin importar qué otros tests se ejecutaron antes.

¿Cuándo necesitamos `tearDown`? Para la clase `Agenda` que solo está en memoria, no es estrictamente necesario — Python va a liberar la memoria al terminar cada test. Pero cuando trabajamos con archivos, conexiones a bases de datos o sockets, sí es crítico cerrarlos en `tearDown` para evitar leaks de recursos.

[python-testing.pdf — slide “La clase TestCase - fixtures”]

4.4.8. El ciclo de vida completo de una clase de tests

Para entender exactamente en qué orden se ejecuta todo, veamos el ciclo completo:

```

setUpClass()      ← 1 vez al inicio de la clase
    setUp()      ← antes de test_A
        test_A()
    tearDown()   ← después de test_A (aunque falle)
        setUp()  ← antes de test_B
            test_B()
        tearDown()
tearDownClass()  ← 1 vez al final de la clase

```

- `setUpClass` y `tearDownClass` se ejecutan una sola vez por clase. Son `classmethod` (reciben `cls`, no `self`). Se usan para recursos costosos de inicializar: conexiones a bases de datos, carga de archivos grandes, etc.
- `setUp` y `tearDown` se ejecutan por cada test individual.

En este módulo, para los tests de la clase Agenda, `setUp` es suficiente. `setUpClass` lo van a ver en acción en el Módulo III con Django, donde inicializar la base de datos de test tiene un costo real.

4.4.9. Ejecución desde la terminal

Los comandos que van a usar en este módulo:

```

# Ejecutar un archivo específico
python -m unittest test_agenda.py

# Con detalle (verbose) - muestra el nombre de cada test
python -m unittest -v test_agenda.py

# Descubrimiento automático - busca test*.py en todo el proyecto
python -m unittest discover

# Descubrimiento en una carpeta específica
python -m unittest discover -s tests/ -p "test_*.py"

# Solo tests que contengan "agregar" en el nombre
python -m unittest -k agregar

# Parar en el primer fallo (útil mientras desarrollamos)
python -m unittest -f test_agenda.py

```

La salida esperada cuando todo anda:

```

test_agregar_nuevo (test_agenda.AgendaTest) ... ok
test_agregar_duplicado (test_agenda.AgendaTest) ... ok
test_buscar_existente (test_agenda.AgendaTest) ... ok

-----
Ran 3 tests in 0.001s

OK

```

4.5. TDD: Test-Driven Development

(Ver Filminas [F-21] a [F-25])

4.5.1. La idea contraintuitiva

TDD (*Test-Driven Development*) es una metodología de diseño formalizada por Kent Beck — sí, el mismo que creó SUnit y JUnit — en su libro *Test-Driven Development by Example* (2002). La idea central es escribir el test **antes** de tener el código que lo hace pasar.

¿Por qué eso mejora el diseño? Porque para escribir un test, necesitás pensar en cómo se va a **usar** el código: cuál es la interfaz pública, qué parámetros recibe, qué retorna, qué excepciones lanza. Eso te fuerza a diseñar la API antes de implementarla. El resultado es código que es naturalmente testeable, porque fue concebido desde la perspectiva de quien lo consume.

El mantra de TDD:

"Never write a single line of production code unless you have a failing test."

— Kent Beck

4.5.2. El ciclo Red → Green → Refactor

TDD se ejecuta en un ciclo de tres pasos que se repite una y otra vez:

● RED — Escribir un test que falla:

- Escribís el test antes de tener el código.
- Lo ejecutás. Debe fallar. Si pasa sin código, el test no está verificando nada real.
- El fallo confirma que el test mide algo que todavía no existe.

● GREEN — Hacer pasar el test:

- Escribís el **mínimo código posible** para que el test pase.
- No optimizés, no generalicés. Si el test espera que `fizzbuzz(3)` retorne `"Fizz"`, podés escribir `return "Fizz"` y el test va a pasar. Eso está bien por ahora.
- El punto es que cada test nuevo te va a forzar a agregar funcionalidad real.

● REFACTOR — Limpiar el código:

- Con los tests en verde, mejorás la implementación: type hints, nombres más claros, eliminar duplicación, extraer constantes.
- Después de cada cambio, ejecutás los tests para verificar que siguen en verde.
- Los tests son la red de seguridad que te permite refactorizar con confianza.

El ciclo completo debería durar entre 2 y 10 minutos. Ciclos cortos, cambios pequeños, feedback constante.

4.5.3. Demo: FizzBuzz con TDD

FizzBuzz es una kata clásica de TDD. La especificación es simple:

- Si `n` es divisible por 3 → retorna `"Fizz"`
- Si `n` es divisible por 5 → retorna `"Buzz"`
- Si es divisible por ambos → retorna `"FizzBuzz"`
- Si no → retorna el número como string

Iteración 1 — ● RED:

```
import unittest

class FizzBuzzTest(unittest.TestCase):

    def test_tres_retorna_fizz(self):
        self.assertEqual(fizzbuzz(3), "Fizz")
```

Ejecutamos: `NameError: name 'fizzbuzz' is not defined`. El test falla. Confirmado: estamos en rojo.

Iteración 1 — ● GREEN:

```
def fizzbuzz(n: int) -> str:
    return "Fizz"
```

Ejecutamos: pasa. Sí, es una implementación tonta. Pero el test está en verde.

Iteración 2 — ● RED:

Agregamos un nuevo test:

```
def test_cinco_retorna_buzz(self):
    self.assertEqual(fizzbuzz(5), "Buzz")
```

Ejecutamos: falla, porque `return "Fizz"` no funciona para 5.

Iteración 2 — ● GREEN:

```
def fizzbuzz(n: int) -> str:
    if n % 3 == 0:
        return "Fizz"
    if n % 5 == 0:
        return "Buzz"
    return str(n)
```

Ahora pasan los dos tests. Seguimos con el caso de 15 (divisible por ambos), y finalmente con un número que no es divisible por ninguno.

Iteración final — ● REFACTOR:

```
def fizzbuzz(n: int) -> str:
    """Retorna Fizz, Buzz, FizzBuzz o el número como string."""
    if n % 15 == 0:
        return "FizzBuzz"
    if n % 3 == 0:
        return "Fizz"
    if n % 5 == 0:
        return "Buzz"
    return str(n)
```

Verificación final:

```
test_cinco_retorna_buzz ... ok
test_quince_retorna_fizzbuzz ... ok
test_tres_retorna_fizz ... ok
test_uno_retorna_uno ... ok

Ran 4 tests in 0.000s OK
```

Fijate el patrón: cada test nuevo nos forzó a agregar funcionalidad real a la función. En ningún momento sobre-implementamos — escribimos exactamente lo que los tests necesitaban. Y al final, los tests documentan la especificación completa de `fizzbuzz`: cualquiera que los lea sabe exactamente qué hace la función, sin necesidad de leer la implementación.

4.6. unittest.mock — aislar dependencias

(Ver *Filminas* [F-30] a [F-33b])

4.6.1. El problema de las dependencias externas

Consideremos este escenario: la clase `Agenda` tiene un método `guardar()` que escribe los contactos en un archivo JSON:

```
class Agenda:
    def guardar(self, path: str) -> None:
        with open(path, "w") as f:
            json.dump(self._contactos, f)
```

Si queremos testear este método, ¿abrimos un archivo real en el disco? Eso presenta varios problemas:

- El test deja archivos basura en el sistema.
- Si el disco está lleno o no tiene permisos, el test falla por razones ajenas al código.
- Es lento comparado con una operación en memoria.
- Si el test falla porque el disco no responde, vamos a creer que hay un bug en `guardar()` cuando en realidad el problema es la infraestructura.

La solución: reemplazar la dependencia real (`open`) con un **doble de test** que simula su comportamiento.

4.6.2. Tipos de dobles de test

No todos los dobles son iguales, aunque en Python coloquialmente se les dice “mocks” a todos:

Tipo	Qué hace	Cuándo usarlo
Stub	Retorna valores fijos	Aislar una fuente de datos
Mock	Registra llamadas + permite verificación	Verificar que un método fue llamado con ciertos argumentos
Spy	Como un mock pero usa la implementación real	Verificar interacciones sin reemplazar la lógica
Fake	Implementación simplificada funcional	Base de datos en memoria

`MagicMock` de Python puede actuar como cualquiera de estos según cómo lo configuremos.

4.6.3. MagicMock

`MagicMock` es un objeto que acepta cualquier atributo y cualquier método sin lanzar errores, y registra todas las llamadas que recibe:

```
from unittest.mock import MagicMock

# Crear el doble
db_falsa = MagicMock()

# Llamar cualquier método - no lanza error
db_falsa.guardar({"dni": "12345678", "nombre": "Juan"})

# Verificar que fue llamado
db_falsa.guardar.assert_called_once()

# Verificar con qué argumentos
db_falsa.guardar.assert_called_with({"dni": "12345678", "nombre": "Juan"})

# Configurar valor de retorno
db_falsa.buscar.return_value = {"nombre": "Juan"}
resultado = db_falsa.buscar("12345678") # → {"nombre": "Juan"}
```

Los métodos de verificación más usados:

Método	Qué verifica
<code>assert_called()</code>	Fue llamado al menos una vez
<code>assert_called_once()</code>	Fue llamado exactamente una vez
<code>assert_called_with(args)</code>	La última llamada usó estos argumentos
<code>assert_called_once_with(args)</code>	Fue llamado una vez con estos argumentos
<code>assert_any_call(args)</code>	En alguna de las llamadas usó estos argumentos
<code>.call_count</code>	Cantidad de veces que fue llamado (propiedad)
<code>.call_args_list</code>	Lista de todas las llamadas con sus argumentos

Para controlar qué devuelve el mock:

- `return_value` : siempre retorna lo mismo. `db.buscar.return_value = {"nombre": "Juan"}` .
- `side_effect` : comportamiento dinámico. Puede ser una secuencia de valores o una excepción: `db.buscar.side_effect = [{"nombre": "Juan"}, KeyError("no encontrado")]` .
- Si configuramos ambos, `side_effect` tiene prioridad sobre `return_value` .

4.6.4. patch — reemplazar en contexto

`patch` es la herramienta que permite reemplazar una función o clase por un doble **solo dentro de un contexto específico**. Cuando el contexto termina, la función original se restaura automáticamente:

```
from unittest.mock import patch, mock_open

class AgendaTest(unittest.TestCase):

    def test_guardar_llama_open(self):
        agenda = Agenda()
        agenda.agregar("12345678", "Juan", "Pérez", "Calle 1", "111")

        with patch("builtins.open", mock_open()) as m:
            agenda.guardar("agenda.json")

        m.assert_called_once_with("agenda.json", "w")
```

`patch("builtins.open", mock_open())` reemplaza la función `open` por un doble que simula la apertura de un archivo. Dentro del `with`, cualquier llamada a `open` va a llegar al doble en lugar de tocar el disco real. Al salir del `with`, `open` vuelve a ser la función real.

La regla más importante de patch: hay que parchear donde se **usa** la función, no donde está **definida**. Si `agenda.py` hace `import json` y luego usa `json.dump(...)`, el path correcto para patch es:

```
# ✅ CORRECTO – parchea donde agenda.py lo usa
with patch("agenda.json.dump") as m: ...

# ❌ INCORRECTO – parchea la definición original
with patch("json.dump") as m: ...
```

La razón: cuando Python importa `json` en el módulo `agenda`, crea una referencia local a `json` dentro del namespace de `agenda`. Si parcheamos `json.dump` directamente, la referencia que `agenda.py` ya tiene no cambia.

`patch` también se puede usar como decorador:

```
@patch("builtins.open", new_callable=mock_open)
def test_guardar_llama_open(self, mock_open_obj):
    agenda = Agenda()
    agenda.agregar("12345678", "Juan", "Pérez", "Calle 1", "111")
    agenda.guardar("agenda.json")
    mock_open_obj.assert_called_once_with("agenda.json", "w")
```

La diferencia es de alcance: el decorador activa el patch para todo el método, mientras que el context manager (`with`) permite activarlo solo en una parte del test.

4.7. Subtests y skipping

(Ver Filminas [F-34] a [F-36b])

4.7.1. subTest — parametrizar sin duplicar código

Cuando queremos probar una función con muchos valores de entrada distintos, tenemos un dilema: ¿escribimos un método `test_*` por cada caso (mucho código repetido) o metemos todo en un solo método (si falla el primero, no vemos los demás)?

`subTest` resuelve esto:

```
def test_fizzbuzz_casos(self):
    casos = [
        (3, "Fizz"),
        (5, "Buzz"),
        (15, "FizzBuzz"),
        (1, "1"),
        (30, "FizzBuzz"),
    ]
    for entrada, esperado in casos:
        with self.subTest(n=entrada):
            self.assertEqual(fizzbuzz(entrada), esperado)
```

Si alguno de los casos falla, los demás **siguen ejecutándose**. El reporte muestra exactamente cuáles fallaron, con el valor del parámetro incluido:

```
FAIL: test_fizzbuzz_casos (n=5)
AssertionError: 'Buzz' != 'WRONG'
```

4.7.2. Skipping — saltar tests condicionalmente

A veces necesitamos que ciertos tests no se ejecuten, por ejemplo porque la funcionalidad aún no está implementada o porque depende de una versión específica de Python:

```
@unittest.skip("Función aún no implementada")
def test_en_construccion(self):
    self.assertTrue(funcion_pendiente())

@unittest.skipIf(sys.version_info < (3, 11), "Requiere Python 3.11+")
def test_solo_en_python_nuevo(self):
    ...

@unittest.expectedFailure
def test_bug_conocido(self):
    # Este test DEBE fallar - si pasa, se reporta como error inesperado
    self.assertEqual(1, 2)
```

`@expectedFailure` es especialmente útil durante TDD: podemos escribir todos los tests al inicio y marcar los de funcionalidades pendientes con este decorador. El suite no falla ruidosamente, pero los tests documentan formalmente qué queda por implementar. Cuando implementamos el método correspondiente, sacamos el decorador y el test pasa a correr normalmente.

4.7.3. addCleanup — limpieza robusta

`addCleanup` es una alternativa a `tearDown` que permite registrar funciones de limpieza desde cualquier punto del `setUp` o del test:

```
def setUp(self):
    self.archivo_temp = crear_archivo_temporal()
    self.addCleanup(os.remove, self.archivo_temp)
```

La ventaja sobre `tearDown` : si `setUp` falla a la mitad, `tearDown` no se ejecuta, pero las funciones registradas con `addCleanup` **sí se ejecutan** (las que ya fueron registradas antes de la falla). Además, se ejecutan en orden LIFO (la última registrada se ejecuta primero).

4.8. Anticipo: django.test.TestCase

(Ver Filminas [F-37] a [F-39])

Para cerrar el módulo, un adelanto de lo que viene en el Módulo III: `django.test.TestCase` **hereda directamente de** `unittest.TestCase` . Esto significa que todo lo que aprendimos acá se transfiere sin cambios:

```
# Lo que aprendimos en este módulo
import unittest

class AgendaTest(unittest.TestCase):
    def setUp(self): ...
    def test_algo(self): self.assertEqual(...)

# Lo que viene en Módulo III – mismo patrón
from django.test import TestCase

class PostTest(TestCase):
    def setUp(self): ...
    def test_algo(self): self.assertEqual(...)
```

Lo que Django agrega encima de `unittest` :

- Un **cliente HTTP de prueba** (`self.client`) para simular requests sin tocar la red.
- Una **base de datos de test** que se crea al inicio y se destruye al final.
- **Rollback automático** entre tests para que la BD quede limpia.

En el TP 3, van a escribir tests para una BlogApp usando este mismo patrón. La transición es directa.

5. Ejemplos trabajados

Ejemplo 1: Escribir tests para la clase Agenda usando TDD

Este ejemplo recorre paso a paso el proceso de crear tests para la clase `Agenda` siguiendo TDD. Es el mismo ejercicio que hacemos en la práctica, pero acá lo explicamos con más detalle.

Paso 1 — Definir qué vamos a testear (sin escribir la Agenda todavía):

Según la especificación, la `Agenda` tiene estos métodos:

Método	Comportamiento esperado	Excepciones
<code>agregar(dni, nombre, apellido, dir, tel)</code>	Registra un contacto	<code>ValueError</code> si DNI inválido, <code>KeyError</code> si duplicado
<code>buscar(dni)</code>	Retorna dict con datos	<code>KeyError</code> si no existe
<code>eliminar(dni)</code>	Elimina el contacto	<code>KeyError</code> si no existe
<code>listar()</code>	Retorna lista de DNIs	—

Paso 2 — RED: Escribir el primer test:

```
import unittest
from agenda import Agenda

class AgendaTest(unittest.TestCase):

    def test_agregar_y_buscar(self):
        agenda = Agenda()
        agenda.agregar("12345678", "Juan", "Pérez", "Calle 1", "2901-111")
        resultado = agenda.buscar("12345678")
        self.assertEqual(resultado["nombre"], "Juan")
        self.assertEqual(resultado["apellido"], "Pérez")
```

Ejecutamos: `ModuleNotFoundError: No module named 'agenda'`. Perfecto, estamos en rojo.

Paso 3 — GREEN: Implementar lo mínimo:

```
# agenda.py
class Agenda:
    def __init__(self):
        self._contactos = {}

    def agregar(self, dni, nombre, apellido, direccion, telefono):
        self._contactos[dni] = {
            "nombre": nombre,
            "apellido": apellido,
            "direccion": direccion,
            "telefono": telefono,
        }

    def buscar(self, dni):
        return self._contactos[dni]
```

Ejecutamos: test en verde. Todavía no hay validaciones, pero el flujo básico funciona.

Paso 4 — RED: Agregar test de validación de DNI:

```
def test_dni_no_numerico_lanza_ValueError(self):
    agenda = Agenda()
    with self.assertRaises(ValueError):
        agenda.agregar("abc", "Juan", "Pérez", "Calle 1", "111")

def test_dni_muy_corto_lanza_ValueError(self):
    agenda = Agenda()
    with self.assertRaises(ValueError):
        agenda.agregar("123", "Juan", "Pérez", "Calle 1", "111")
```

Ejecutamos: fallan, porque `agregar` no tiene validaciones todavía.

Paso 5 — GREEN: Agregar validaciones:

```
def agregar(self, dni, nombre, apellido, direccion, telefono):
    if not dni.isdigit() or not (7 <= len(dni) <= 8):
        raise ValueError(f"DNI inválido: {dni!r}")
    if dni in self._contactos:
        raise KeyError(f"DNI {dni!r} ya registrado")
    self._contactos[dni] = {
        "nombre": nombre,
        "apellido": apellido,
        "direccion": direccion,
        "telefono": telefono,
    }
```

Ejecutamos: todo en verde.

Paso 6 — REFACTOR: Agregar setUp para evitar repetición:

```
class AgendaTest(unittest.TestCase):

    def setUp(self):
        self.agenda = Agenda()
        self.agenda.agregar("12345678", "Juan", "Pérez", "Calle 1", "2901-111")
        self.agenda.agregar("87654321", "Ana", "García", "Calle 2", "2901-222")

    def tearDown(self):
        del self.agenda

    def test_buscar_existente(self):
        resultado = self.agenda.buscar("12345678")
        self.assertEqual(resultado["nombre"], "Juan")

    def test_buscar_inexistente_lanza_KeyError(self):
        with self.assertRaises(KeyError):
            self.agenda.buscar("00000000")

    def test_agregar_duplicado_lanza_KeyError(self):
        with self.assertRaises(KeyError):
            self.agenda.agregar("12345678", "Otro", "Nombre", "Calle 3", "111")

    def test_dni_no_numerico_lanza_ValueError(self):
        with self.assertRaises(ValueError):
            self.agenda.agregar("abc", "Juan", "Pérez", "Calle 1", "111")

    def test_dni_muy_corto_lanza_ValueError(self):
        with self.assertRaises(ValueError):
            self.agenda.agregar("123", "Juan", "Pérez", "Calle 1", "111")
```

Notar cómo cada test tiene un nombre descriptivo: `test_buscar_inexistente_lanza_KeyError` te dice exactamente qué escenario prueba y qué espera que ocurra. Eso es mucho mejor que `test_1` o `test_buscar`.

Ejemplo 2: FizzBuzz completo con subTest

Este ejemplo muestra cómo combinar TDD con `subTest` para testear múltiples entradas de forma compacta:

```
import unittest

def fizzbuzz(n: int) -> str:
    """Retorna Fizz, Buzz, FizzBuzz o el número como string."""
    if n % 15 == 0:
        return "FizzBuzz"
    if n % 3 == 0:
        return "Fizz"
    if n % 5 == 0:
        return "Buzz"
    return str(n)

class FizzBuzzTest(unittest.TestCase):

    def test_multiplos_de_tres(self):
        """Todos los múltiplos de 3 (que no sean de 15) retornan Fizz."""
        for n in [3, 6, 9, 12, 18, 21]:
            with self.subTest(n=n):
                self.assertEqual(fizzbuzz(n), "Fizz")

    def test_multiplos_de_cinco(self):
        """Todos los múltiplos de 5 (que no sean de 15) retornan Buzz."""
        for n in [5, 10, 20, 25]:
            with self.subTest(n=n):
                self.assertEqual(fizzbuzz(n), "Buzz")

    def test_multiplos_de_quince(self):
        """Divisibles por 3 y por 5 retornan FizzBuzz."""
        for n in [15, 30, 45, 60]:
            with self.subTest(n=n):
                self.assertEqual(fizzbuzz(n), "FizzBuzz")

    def test_numeros_normales(self):
        """Números no divisibles por 3 ni 5 retornan el número como string."""
        for n in [1, 2, 4, 7, 11, 13]:
            with self.subTest(n=n):
                self.assertEqual(fizzbuzz(n), str(n))

    def test_caso_borde_cero(self):
        """Cero es divisible por 15 – retorna FizzBuzz."""
        self.assertEqual(fizzbuzz(0), "FizzBuzz")

    def test_negativos(self):
        """Los negativos también siguen las reglas de divisibilidad."""
        self.assertEqual(fizzbuzz(-3), "Fizz")
        self.assertEqual(fizzbuzz(-5), "Buzz")
        self.assertEqual(fizzbuzz(-15), "FizzBuzz")
```

Fijate que:

- Los tests están agrupados por categoría (múltiplos de 3, de 5, de 15, normales).
- `subTest` permite probar muchos valores sin duplicar código.
- Se incluyen casos de borde (cero, negativos) — justamente el tipo de caso que detectaría bugs como el del Ariane 5.

Ejemplo 3: Testear un método que usa archivos (mock)

```
import json
import unittest
from unittest.mock import patch, mock_open, MagicMock

class Agenda:
    def __init__(self):
        self._contactos = {}

    def agregar(self, dni, nombre, apellido, direccion, telefono):
        if not dni.isdigit() or not (7 <= len(dni) <= 8):
            raise ValueError(f"DNI inválido: {dni!r}")
        self._contactos[dni] = {
            "nombre": nombre, "apellido": apellido,
            "direccion": direccion, "telefono": telefono,
        }

    def guardar(self, path: str) -> None:
        with open(path, "w") as f:
            json.dump(self._contactos, f)

    def cargar(self, path: str) -> None:
        with open(path, "r") as f:
            self._contactos = json.load(f)

class AgendaPersistenciaTest(unittest.TestCase):
    """Tests de persistencia – sin tocar el disco real."""

    def setUp(self):
        self.agenda = Agenda()
        self.agenda.agregar("12345678", "Juan", "Pérez", "Calle 1", "111")

    def test_guardar_abre_archivo_correcto(self):
        """Verificamos que guardar() llama a open con el path correcto."""
        with patch("builtins.open", mock_open()) as m:
            self.agenda.guardar("agenda.json")
            m.assert_called_once_with("agenda.json", "w")

    @patch("builtins.open", new_callable=mock_open,
           read_data='{"12345678": {"nombre": "Juan"}}')
    def test_cargar_restaura_contactos(self, m):
        """Verificamos que cargar() lee el archivo y restaura el dict."""
        agenda_nueva = Agenda()
        # patch de json.load para que retorne un dict controlado
        with patch("json.load", return_value={"12345678": {"nombre": "Juan"}}):
            agenda_nueva.cargar("agenda.json")
            self.assertIn("12345678", agenda_nueva._contactos)
            self.assertEqual(agenda_nueva._contactos["12345678"]["nombre"], "Juan")
```

Puntos a notar:

- En ningún momento se crea un archivo real en el disco.
- `mock_open()` simula la apertura de un archivo.
- Usamos `assert_called_once_with` para verificar que `open` fue llamado con los argumentos correctos.
- En el test de `cargar`, controlamos exactamente qué “lee” el mock del archivo.

6. Puntos clave y resumen

Mapa conceptual del módulo

```
PRUEBAS UNITARIAS
├── ¿POR QUÉ TESTEAR?
│   ├── Casos reales: Air China, Therac-25, Ariane 5
│   ├── Testing = falsificación (buscar dónde falla)
│   └── Pruebas de regresión = red de seguridad
├── CLASIFICACIÓN
│   ├── Por conocimiento: caja negra vs. caja blanca
│   └── Por nivel: unidad → integración → sistema → aceptación
├── FRAMEWORK unittest
│   ├── Familia xUnit (Beck, 1989 → Python 2001)
│   ├── TestCase → TestSuite → TestRunner
│   ├── Aserciones: assertEquals, assertRaises, assertIn...
│   ├── Fixtures: setUp / tearDown / setUpClass
│   └── CLI: python -m unittest discover
├── TDD
│   ├── Ciclo: RED → GREEN → REFACTOR
│   ├── Test primero, implementación después
│   └── Los tests son especificación ejecutable
├── MOCKING
│   ├── MagicMock: doble que acepta todo y registra llamadas
│   ├── patch: reemplazar dependencias en contexto
│   └── Regla: parchear donde se USA, no donde se DEFINE
└── EXTRAS
    ├── subTest: parametrizar sin duplicar
    ├── skip / expectedFailure: manejo de tests pendientes
    └── addCleanup: limpieza robusta
```

Los 10 puntos que no podés olvidar

1. **Testing es falsificación:** el objetivo no es confirmar que funciona, sino encontrar dónde falla.
2. **Cada test debe ser independiente:** no depender del orden de ejecución ni del estado de otros tests.
3. **assertEquals > assertTrue:** siempre usá la aserción semántica más específica disponible.
4. **assertRaises con with:** es la forma estándar de verificar que se lanza una excepción.
5. **setUp corre antes de CADA test:** garantiza estado limpio e independencia.
6. **TDD = Red → Green → Refactor:** escribir el test antes del código, implementar lo mínimo, limpiar.
7. **Los nombres de tests son documentación:** `test_agregar_dni_invalido_lanza_ValueError` se explica solo.
8. **patch parchea donde se USA:** si `agenda.py` importa `json`, el path es `"agenda.json.dump"`.
9. **python -m unittest discover es tu amigo:** descubre y ejecuta todos los tests del proyecto automáticamente.
10. **Todo se transfiere a Django:** `django.test.TestCase` hereda de `unittest.TestCase`.

7. Autoevaluación

Estas preguntas son para que verifiques tu comprensión del módulo **antes** de hacer el TP. Son distintas a las consignas del trabajo práctico — apuntan a conceptos, no a implementación.

Pregunta 1 — Conceptual

¿Cuál es la diferencia entre un resultado **FAIL** y un resultado **ERROR** en la salida de `unittest` ? Dá un ejemplo concreto de cada uno usando la clase `Agenda`.

► Orientación

Pregunta 2 — Análisis

Dado el siguiente test, ¿es de caja negra o de caja blanca? Justificá.

```
def test_dni_seis_digitos_es_invalido(self):
    with self.assertRaises(ValueError):
        self.agenda.agregar("123456", "Test", "Test", "Dir", "Tel")
```

► Orientación

Pregunta 3 — Aplicación

Escribí un test unitario (sin ejecutarlo, solo el código) que verifique que el método `listar()` de la `Agenda` retorna una lista que contiene todos los DNIs agregados, sin importar el orden.

► Orientación

Pregunta 4 — Diseño

¿Por qué es importante que `setUp` se ejecute antes de **cada** test y no una sola vez al principio? Pensá en un escenario con la `Agenda` donde ejecutar `setUp` una sola vez causaría problemas.

► Orientación

Pregunta 5 — Mocking

En el siguiente código, ¿por qué el path del patch es `"agenda.json.dump"` y no `"json.dump"` ?

```
# archivo: agenda.py
import json

class Agenda:
    def guardar(self, path):
        json.dump(self._contactos, open(path, "w"))
```

```
with patch("agenda.json.dump") as mock_dump:
    agenda.guardar("out.json")
```

► Orientación

Pregunta 6 — TDD

Ordená los siguientes pasos del ciclo TDD: a) Mejorar los nombres de variables y agregar type hints. b) Escribir `def test_eliminar_existente(self)` con un assert. c) Implementar el método `eliminar()` en la clase Agenda. d) Ejecutar el test y verificar que falla. e) Ejecutar el test y verificar que pasa.

► Orientación

Pregunta 7 — Integración de conceptos

¿Qué tipo de prueba habría detectado el fallo del Ariane 5? ¿De caja negra, caja blanca, o ambas? ¿Y a qué nivel de la pirámide de testing corresponde?

► Orientación

Pregunta 8 — Reflexión

Un compañero escribe el siguiente test y dice “está en verde, todo funciona”:

```
def test_agenda(self):  
    agenda = Agenda()  
    agenda.agregar("12345678", "Juan", "Pérez", "Calle 1", "111")
```

¿Qué problema tiene este test? ¿Qué “test smell” presenta?

► Orientación

8. Glosario

Término	Definición
Aserción	Verificación dentro de un test que compara un resultado obtenido contra un resultado esperado. Si no se cumple, el test falla. En <code>unittest</code> , los métodos <code>assert*</code> de <code>TestCase</code> .
Caja blanca	Enfoque de testing donde se conoce la estructura interna del código y se diseñan las entradas para ejercitar ramas y condiciones específicas.
Caja negra	Enfoque de testing donde se verifican entradas y salidas contra la especificación, sin conocer la implementación interna.
Caso de borde (<i>boundary case</i>)	Entrada que está en el límite exacto de una condición. Por ejemplo, un DNI de 7 dígitos cuando la validación es <code>7 <= len(dni) <= 8</code> .
Cobertura (<i>coverage</i>)	Porcentaje del código fuente que fue ejercitado por los tests. 100% de cobertura no garantiza corrección, pero baja cobertura garantiza que hay código sin testear.
Doble de test (<i>test double</i>)	Objeto que reemplaza una dependencia real durante un test. Incluye stubs, mocks, spies y fakes.
Fixture	Código de preparación y limpieza que se ejecuta antes y después de los tests. En <code>unittest</code> : <code>setUp</code> , <code>tearDown</code> , <code>setUpClass</code> , <code>tearDownClass</code> .
MagicMock	Clase de <code>unittest.mock</code> que crea un objeto que acepta cualquier atributo o método, registra todas las llamadas, y permite configurar valores de retorno.
Mock	En sentido estricto, un doble de test que registra las llamadas recibidas y permite verificarlas. En uso coloquial en Python, cualquier doble de test.
Overflow	Error que ocurre cuando un valor numérico excede el rango que puede representar el tipo de dato. Causa del fallo del Ariane 5.
Patch	Técnica de reemplazo temporal de una función, clase o módulo por un doble, limitada a un contexto específico. En <code>unittest.mock</code> : <code>patch()</code> .
Prueba de regresión	Re-ejecución de toda la batería de tests después de un cambio, para verificar que no se rompió funcionalidad existente.
Prueba de unidad (<i>unit test</i>)	Test que verifica una clase o función aislada, sin dependencias externas reales. Nivel 1 de la pirámide de testing.
Race condition	Situación donde el resultado depende del orden temporal de eventos no determinísticos. Causa del fallo del Therac-25.
Runner	Componente que ejecuta los tests y reporta los resultados. En <code>unittest</code> : <code>TextTestRunner</code> , invocado con <code>python -m unittest</code> .
subTest	Mecanismo de <code>unittest</code> que permite ejecutar múltiples verificaciones dentro de un solo método de test, reportando cada fallo individualmente.

Término	Definición
TDD (<i>Test-Driven Development</i>)	Metodología de desarrollo donde el test se escribe antes que el código de producción. Ciclo: Red (test falla) → Green (código mínimo para pasar) → Refactor (limpiar).
TestCase	Clase base de <code>unittest</code> de la que heredan todas las clases de test. Provee las aserciones y los fixtures.
TestSuite	Colección de casos de prueba que se ejecutan juntos. Se construye automáticamente por discovery.
xUnit	Familia de frameworks de testing que siguen el patrón de SUnit (Smalltalk, 1989). Incluye JUnit (Java), NUnit (.NET), unittest (Python).

9. Referencias y lecturas recomendadas

Fuentes utilizadas en este módulo

- **Filminas originales del curso** — `python-testing.pdf` (Laboratorio de Programación y Lenguajes, UNTDF 2025). Cubren: errores históricos, definición de testing, tipos de pruebas, framework unittest, fixtures, aserciones, y el ejercicio de la clase Agenda.
- **Documentación oficial de Python** — `unittest` — [Unit testing framework](#). Referencia completa de TestCase, aserciones, fixtures, discovery y CLI. Python 3.14.
- **Documentación oficial de Python** — `unittest.mock` — [Mock object library](#). MagicMock, patch, return_value, side_effect.
- Beck, K. (2002). *Test-Driven Development by Example*. Addison-Wesley. El libro que formalizó TDD como metodología de diseño de software.
- Leveson, N. G., & Turner, C. S. (1993). An investigation of the Therac-25 accidents. *IEEE Computer*, 26(7), 18–41. Análisis detallado de los incidentes del Therac-25.
- Lions, J. L. (1996). *Ariane 5 Flight 501 Failure: Report by the Inquiry Board*. European Space Agency. Informe oficial sobre la falla del Ariane 5.

Lecturas complementarias (opcionales)

- Martin, R. C. (2008). *Clean Code*. Prentice Hall. Capítulo 9: “Unit Tests” — principios FIRST para tests (Fast, Independent, Repeatable, Self-validating, Timely).
- Documentación de `pytest` : [docs.pytest.org](#). Framework alternativo a `unittest` . No lo usamos en este módulo, pero es ampliamente utilizado en la industria.
- Documentación de Django Testing: [docs.djangoproject.com/en/5.1/topics/testing/](#). Referencia para el Módulo III.

Guía de estudio generada para: Laboratorio de Programación y Lenguajes (IF009) — UNTDF 2026

Tema 02 — Pruebas Unitarias · Semanas 4–5

Correspondencia: [filminas.md](#) [F-00] a [F-41] · [minuta.md](#) (semanas 4–5)